

Cloud Cost Decision Agent

1. Project Overview

This project builds a small decision-making agent for cloud cost monitoring.

The main problem is:

The agent observes cloud cost, usage, and recent operational information. It must select whether to wait, investigate, request more evidence, or escalate because the cause of an unusual cost increase is not known.

The main challenge is that an increase in cloud cost does **not always mean there is a problem**.

For example:

- Cloud cost may increase because customer traffic increased.
- A scheduled workload may normally run at the end of the month.
- A deployment may accidentally increase compute usage.
- A cloud resource may have been left running.
- A system may simply have normal day-to-day variation.

The agent therefore needs to reason under uncertainty before selecting an action.

2. Hidden States

The real cause of the cost change is called the **hidden state**.

The agent currently considers four possible hidden states:

`normal`

The cost change is normal variation and does not require investigation.

`expected_pattern`

The cost increase is caused by known recurring behaviour, such as a scheduled batch process.

`legitimate_growth`

The cost increase is caused by useful growth in workload, traffic, customers, or usage.

cost_incident

The increase may be caused by unintended or inefficient cloud spending.

Examples include:

- resources left running
 - over-provisioned compute
 - unexpected logging
 - configuration problems
 - deployment-related problems
-

3. Agent Actions

The agent currently has four possible actions:

WAIT

The available evidence does not suggest an immediate problem.

GET_MORE_EVIDENCE

The situation is suspicious, but the agent is not confident enough to escalate.

ASK_HUMAN

The probability of an incident is high enough that a human should review the case.

ESCALATE

The agent believes there is strong evidence of a cloud cost incident.

4. Project Structure

The current project structure is:

```
student-project/  
|  
├─ src/  
|   ├─ features.py  
|   ├─ belief.py  
|   └─ policy.py
```

```
|   └─ agent.py
|
|   └─ data/
|       └─ test_cases.csv
|
|   └─ experiments/
|       └─ evaluate.py
|
|   └─ results/
|       └─ predictions.csv
|
|   └─ README.md
```

Each file has a different responsibility.

5. features.py

The purpose of `features.py` is to convert raw cloud information into useful signals.

A sample input case looks like this:

```
case = {
    "current_cost": 1450,
    "baseline_cost": 1000,
    "current_usage": 105000,
    "baseline_usage": 100000,
    "recent_deployment": True,
    "expected_pattern": False
}
```

For this case, the feature calculations are:

```
Cost change:      +45.00%
Usage change:     +5.00%
Current unit cost: 0.01381
Baseline unit cost: 0.01000
Unit-cost change: +38.10%
```

Cost Change

Cost change is calculated as:

$$(\text{current cost} - \text{baseline cost}) / \text{baseline cost} \times 100$$

For the example:

$$\begin{aligned} & (1450 - 1000) / 1000 \times 100 \\ & = 45\% \end{aligned}$$

This means cloud cost increased by 45%.

Usage Change

Usage change is calculated as:

$$(\text{current usage} - \text{baseline usage}) / \text{baseline usage} \times 100$$

For the example:

$$\begin{aligned} & (105000 - 100000) / 100000 \times 100 \\ & = 5\% \end{aligned}$$

Usage increased by only 5%.

This is important because cost increased much faster than usage.

Unit Cost

Unit cost is:

$$\text{cloud cost} / \text{usage}$$

For the current period:

$$\begin{aligned} & 1450 / 105000 \\ & = 0.01381 \end{aligned}$$

For the baseline:

$$\begin{aligned} &1000 / 100000 \\ &= 0.01000 \end{aligned}$$

The unit cost therefore increased by approximately:

$$38.10\%$$

This provides more information than cost alone.

If cloud cost and workload both increase by similar percentages, the increase may represent legitimate growth.

If cloud cost increases strongly while workload remains almost unchanged, the situation may be more suspicious.

6. belief.py

The agent does not immediately decide that something is an incident.

Instead, it maintains a probability for each possible hidden state.

The initial belief is:

```
prior_belief = {  
    "normal": 0.25,  
    "expected_pattern": 0.25,  
    "legitimate_growth": 0.25,  
    "cost_incident": 0.25  
}
```

This means that before examining evidence, the agent gives each explanation a probability of 25%.

The probabilities always sum to:

$$1.0 = 100\%$$

7. Updating Beliefs

The agent changes its beliefs when new evidence is observed.

This represents the human reasoning function:

Change a belief after receiving new evidence.

The current version uses simple hand-written rules.

These rules are intentionally simple because this is the first testable version of the agent.

Evidence 1: Expected Pattern

For the example case:

```
expected_pattern = False
```

The initial belief was:

normal	25%
expected pattern	25%
legitimate growth	25%
cost incident	25%

Because the cost change does not match a known expected pattern, the belief changes to approximately:

normal	30.0%
expected pattern	15.0%
legitimate growth	27.5%
cost incident	27.5%

The probability of `expected_pattern` decreases because the evidence says that the current behaviour is not known to be recurring.

Evidence 2: Cost and Usage

The example has:

Cost change = +45%
Usage change = +5%

The agent currently uses the rule:

If cost increases strongly
and usage increases only slightly,
increase the probability of a cost incident.

The belief becomes:

normal	25.0%
expected pattern	12.5%
legitimate growth	25.0%
cost incident	37.5%

The cost incident becomes the most likely explanation.

Evidence 3: Recent Deployment

The example also has:

recent_deployment = **True**

A recent deployment does not prove that an incident occurred.

However, if a large cost increase happens near a deployment, it provides additional evidence that an operational change may have affected cloud spending.

After this evidence, the belief becomes:

normal	20.0%
expected pattern	10.0%
legitimate growth	22.5%
cost incident	47.5%

The agent now believes there is a:

47.5% probability of a cost incident.

The agent is suspicious, but still uncertain.

8. Important Limitation of the Belief Model

The probability changes used in the current version are **not learned from real cloud data**.

For example:

```
+0.10 to cost_incident  
-0.05 from normal
```

are currently design assumptions.

They are being used to create the smallest testable version of the agent.

Later stages of the project should investigate whether these values are realistic using:

- practitioner feedback
- historical cloud incidents
- experiments
- calibration analysis

These values should therefore not currently be presented as real-world probabilities.

9. `policy.py`

Once the agent has calculated its belief, it must select an action.

The current V1 policy uses the probability of `cost_incident`.

```
P(cost incident) < 0.30  
→ WAIT  
  
0.30 ≤ P(cost incident) < 0.50  
→ GET_MORE_EVIDENCE  
  
0.50 ≤ P(cost incident) < 0.70  
→ ASK_HUMAN
```

$P(\text{cost incident}) \geq 0.70$
→ ESCALATE

For the example case:

$P(\text{cost incident}) = 0.475$

Therefore:

$0.30 \leq 0.475 < 0.50$

and the selected action is:

GET_MORE_EVIDENCE

The agent does not immediately escalate because its confidence is not yet high enough.

10. `agent.py`

`agent.py` connects the different components together.

The complete flow is:

```
Input case
↓
Calculate features
↓
Start with prior belief
↓
Update belief using expected pattern
↓
Update belief using cost and usage
↓
Update belief using deployment information
↓
Apply decision policy
↓
Select action
```

The main function is:

```
run_agent(case)
```

It returns:

```
belief  
action
```

For the example case, the output is:

Belief:

normal	0.20
expected_pattern	0.10
legitimate_growth	0.225
cost_incident	0.475

Action:

GET_MORE_EVIDENCE

11. Initial Test Cases

The agent was initially tested using five simulated cases.

The cases represent different hidden states:

```
Case 1 → Cost incident  
Case 2 → Normal  
Case 3 → Legitimate growth  
Case 4 → Expected pattern  
Case 5 → Cost incident
```

The true state is stored in the test data only for evaluation.

The agent should not use `true_state` when making its decision.

12. Initial Agent Results

The current V1 agent produced:

```
Case 1
True state: cost_incident
Action: GET_MORE_EVIDENCE

Case 2
True state: normal
Action: WAIT

Case 3
True state: legitimate_growth
Action: WAIT

Case 4
True state: expected_pattern
Action: GET_MORE_EVIDENCE

Case 5
True state: cost_incident
Action: GET_MORE_EVIDENCE
```

These are only early tests.

Five cases are not enough to make conclusions about agent performance.

The final experiment will require 30–50 cases.

13. Predicted State

In addition to selecting an action, the agent records the hidden state with the highest probability.

This is calculated using:

```
predicted_state = max(belief, key=belief.get)
```

The results are saved to:

```
results/predictions.csv
```

The file contains information such as:

```
case_id
true_state
predicted_state
action
normal_prob
expected_pattern_prob
legitimate_growth_prob
cost_incident_prob
```

This allows the agent's decisions to be analysed later.

14. Baseline Policy

A simple baseline was also created.

The baseline does not reason about uncertainty.

It only looks at cost change.

The rule is:

```
If cost increases by more than 30%
→ ESCALATE

Otherwise
→ WAIT
```

The baseline produced:

```
Case 1: +45% → ESCALATE
Case 2:  +4% → WAIT
Case 3: +50% → ESCALATE
Case 4: +35% → ESCALATE
Case 5: +60% → ESCALATE
```

This shows an important weakness of a simple threshold.

For example:

Case 3
True state = legitimate growth
Baseline = ESCALATE

and:

Case 4
True state = expected pattern
Baseline = ESCALATE

The baseline escalates these cases because it only knows that cost increased.

It does not understand why the cost increased.

15. Why the Agent May Be Better Than the Baseline

The baseline asks:

Did cloud cost increase by more than 30%?

The agent asks:

How much did cost increase?

Did usage increase at the same time?

Did unit cost become worse?

Is this a known expected pattern?

Was there a recent operational change?

How certain am I that this is actually an incident?

This gives the agent more context before making a decision.

The experiment will test whether this additional reasoning actually improves decisions.

It should not simply be assumed that the more complicated agent is better.

16. Current Research Question

A possible research question emerging from the current implementation is:

Can a belief-based cloud cost agent reduce unnecessary escalations compared with a simple cost-threshold policy while still identifying potentially costly incidents?

This question will be refined as more practitioner feedback and experiment results become available.

17. Current Human Reasoning Function

The current agent implements one simple human-like reasoning capability:

Change beliefs when new evidence becomes available.

For example:

```
Initial incident probability
25%

No expected pattern
↓
27.5%

Large cost increase with small usage increase
↓
37.5%

Recent deployment
↓
47.5%
```

The agent does not try to reproduce complete human reasoning.

It performs only a small reasoning task that can be tested.

18. Current Limitations

The current system is only Agent V1.

Known limitations include:

- Belief updates are manually designed.
- Decision thresholds are manually selected.
- Only five initial test cases have been evaluated.
- Test cases are simulated rather than collected from real cloud systems.
- A deployment does not necessarily cause a cost incident.
- The agent currently uses limited operational context.
- Historical similarity has not yet been implemented.
- Costs of incorrect decisions have not yet been added.
- Probability calibration has not yet been tested.
- Reddit and X practitioner feedback has not yet been incorporated into the agent.

These limitations will be addressed or discussed in later stages.

19. Running the Agent

From the project root directory, run:

```
python src/agent.py
```

This runs the agent on the test cases and produces predictions.

The prediction results are saved to:

```
results/predictions.csv
```

20. Running the Baseline Evaluation

Run:

```
python experiments/evaluate.py
```

The baseline currently checks whether cost increased by more than 30%.

It then selects:

ESCALATE

or:

WAIT

The baseline will later be compared with the agent using more complete evaluation metrics.

21. Next Steps

The next stage of the project is to expand the experiment from five cases to approximately 40 simulated cases.

The planned dataset will contain examples of:

Normal variation
Expected patterns
Legitimate growth
Cost incidents

After that, the project will compare:

Baseline threshold policy
vs.
Belief-based Agent V1

The evaluation will include more than simple accuracy.

Possible measurements include:

False positives
False negatives
Human-review rate
Decision cost
Confusion matrix
Calibration

At least five incorrect decisions will also be examined and given named failure conditions.

22. Current Status

Completed so far:

- ✓ Problem selected
- ✓ Hidden states defined
- ✓ Actions defined
- ✓ Feature calculations created
- ✓ Prior belief defined
- ✓ Evidence-based belief updates created
- ✓ Initial decision policy created
- ✓ Agent components connected
- ✓ Five simulated cases created
- ✓ Predictions saved
- ✓ Simple baseline created
- ✓ Initial baseline comparison performed

Still to do:

- Expand to 30-50 cases
- Evaluate two agent policies
- Calculate decision costs
- Analyse incorrect decisions
- Add probability decision record
- Incorporate Reddit/X feedback
- Perform AI reviews
- Write the IJCAI-style preprint
- Prepare final social posts

This README describes the current implementation only. The agent design, probability model, and policies may change based on experiments and practitioner feedback.